
Simulateur de CPU

Depot partiel 1

Document de Conception

Ce document présente l'ensemble des diagrammes UML
réalisés dans le cadre du projet Simulateur de CPU.

Groupe: Dorsa KHOSHNOOD - Amirmahdi GHASEMI

Contenu :

Diagramme de classes · Diagramme de cas d'utilisation
Diagrammes de séquence (Étapes 1 à 5)

1. Diagramme de Classes

Ce diagramme présente l'architecture statique du simulateur de CPU : les classes principales (CPU, Mémoire, BanqueRegistres, ALU, Assembleur, Programme, Instruction, Opérande...), leurs attributs, méthodes et relations (associations, compositions, énumérations).

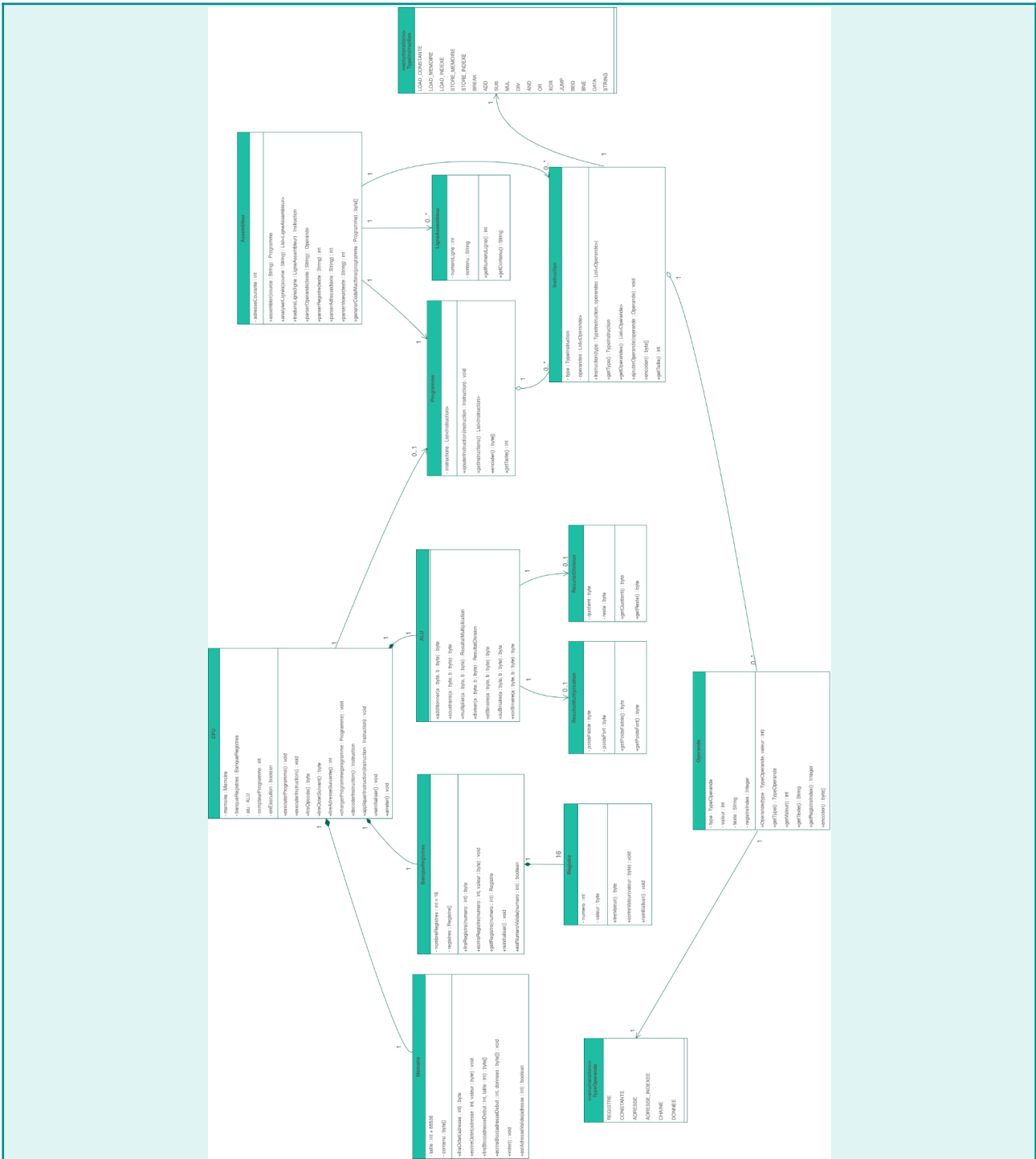


Figure 1 — Diagramme de classes UML du Simulateur de CPU

2. Diagramme de Cas d'Utilisation

Ce diagramme modélise les interactions entre l'utilisateur et le simulateur. Il identifie les cas d'utilisation principaux (exécuter le programme, charger le programme, consulter/modifier la mémoire et les registres) ainsi que leurs relations d'inclusion et d'extension.

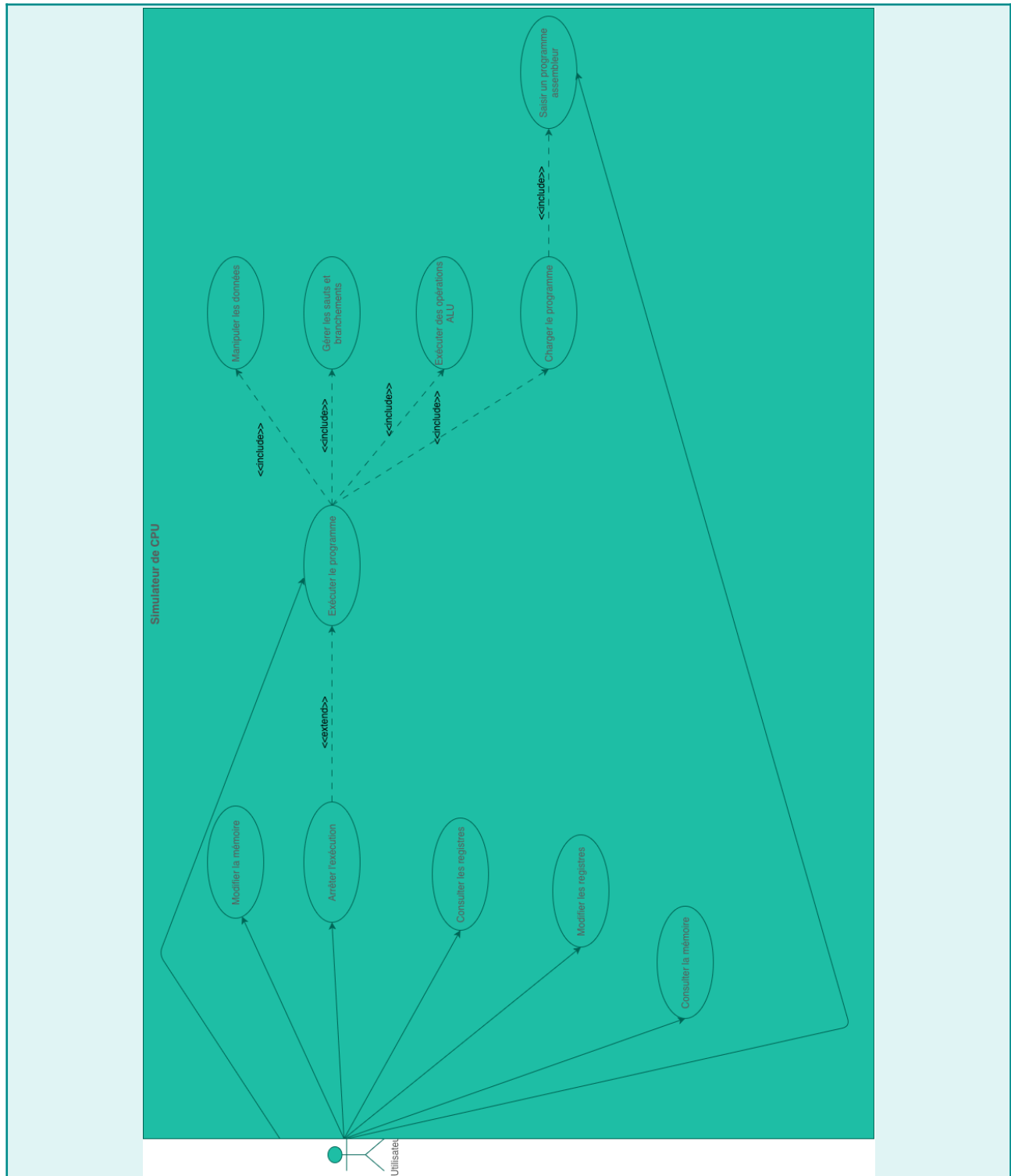


Figure 2 — Diagramme de cas d'utilisation du Simulateur de CPU

3. Diagrammes de Séquence

Les diagrammes de séquence décrivent le comportement dynamique du simulateur à travers cinq étapes progressives, couvrant l'ensemble du jeu d'instructions.

3.1 Étape 1 — Exécution d'un programme (Load / Store / Break)

Ce diagramme illustre le cycle fetch-decode-execute pour les instructions de base : LOAD_CONSTANTE (chargement d'une valeur immédiate dans un registre), LOAD_MEMOIRE (chargement depuis une adresse mémoire), STORE (écriture en mémoire) et BREAK (arrêt de la boucle d'exécution).

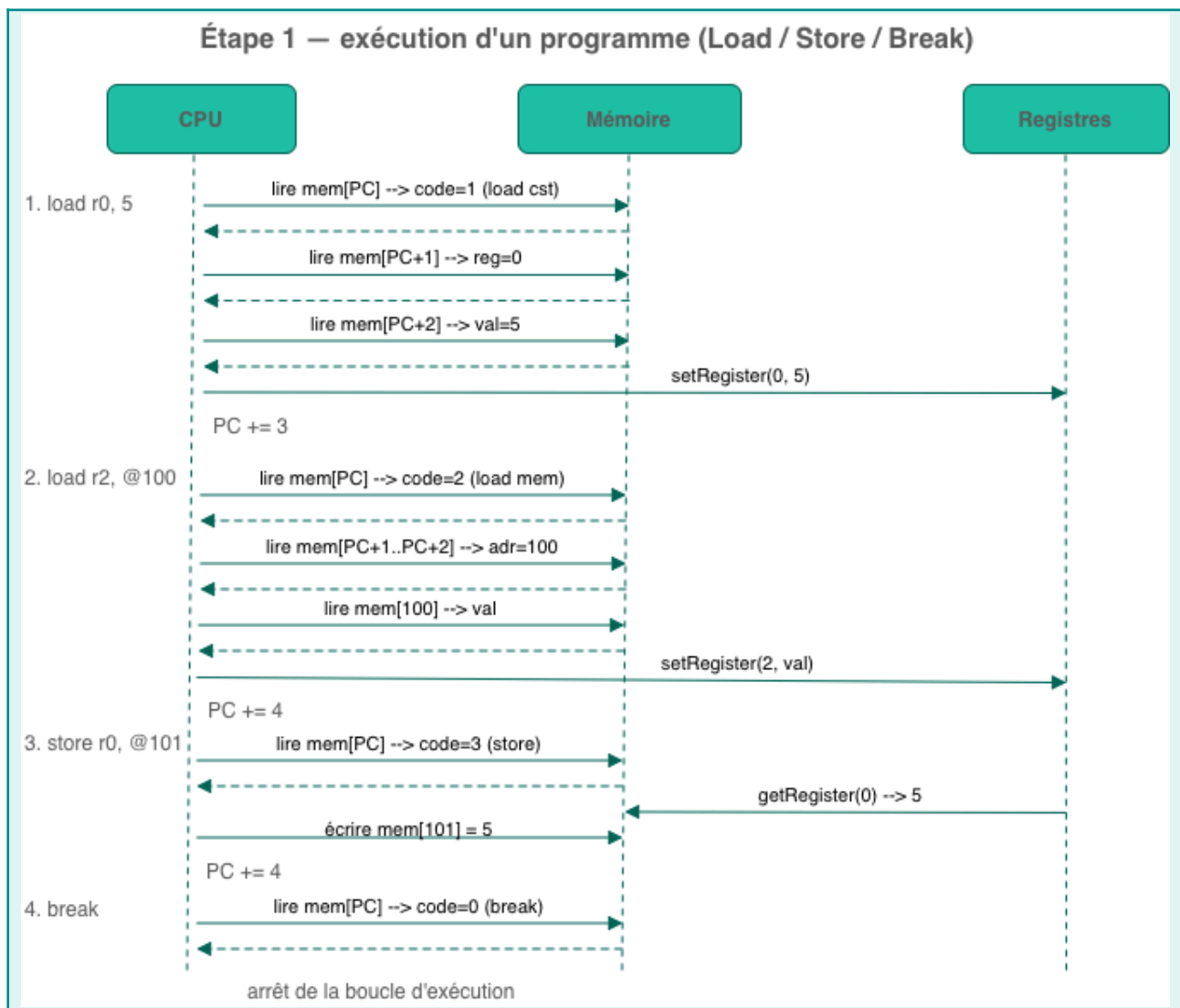


Figure 3 — Séquence Étape 1 : Load / Store / Break

3.2 Étape 2 — Assemblage d'un programme source

Ce diagramme montre le processus d'assemblage : l'utilisateur soumet un code source en assembleur, l'Assembleur analyse chaque ligne (parsing, conversion en opcodes, résolution des adresses hexadécimales), écrit le code machine en mémoire, puis le CPU exécute le programme via son cycle fetch/decode/execute.

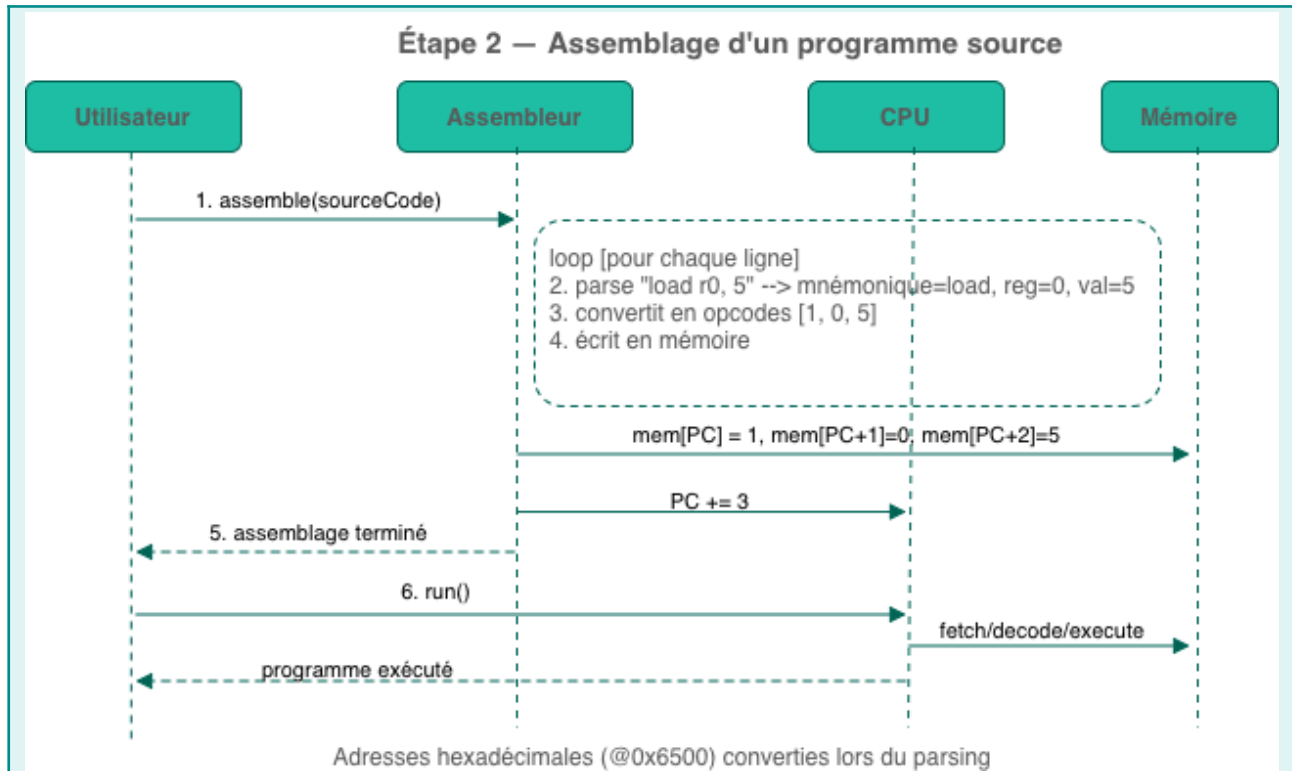


Figure 4 — Séquence Étape 2 : Assemblage d'un programme source

3.3 Étape 3 — Opérations ALU (ADD, MUL, DIV, OR, AND, XOR...)

Ce diagramme détaille l'exécution des instructions arithmétiques et logiques. Le CPU récupère les opérandes depuis les registres, délègue le calcul à l'ALU (ADD, SUB, MUL, DIV, AND, OR, XOR), puis stocke le résultat dans le registre destination. Les instructions MUL et DIV retournent deux valeurs (quotient/reste ou poids fort/faible).

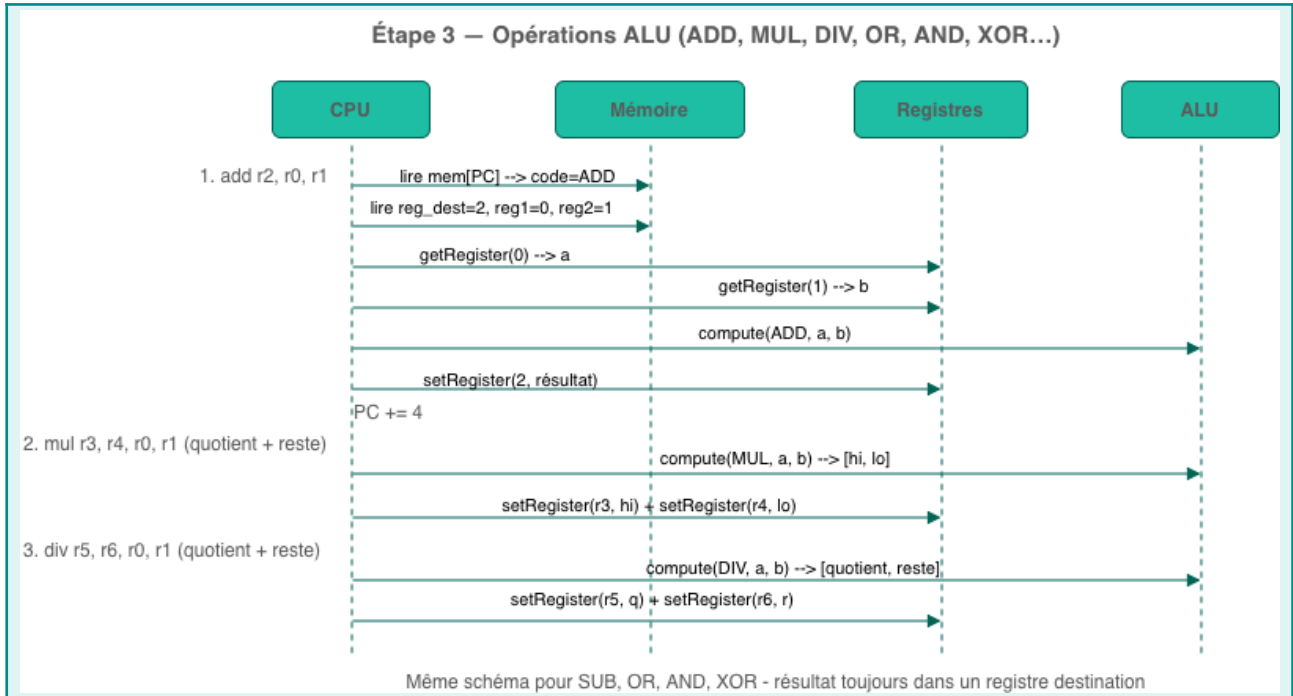


Figure 5 — Séquence Étape 3 : Opérations ALU

3.4 Étape 4 — Sauts et branchements (JUMP / BEQ / BNE)

Ce diagramme représente les instructions de contrôle de flux : JUMP (saut inconditionnel vers une adresse), BEQ (branchement si égalité entre deux registres) et BNE (branchement si inégalité). L'ALU effectue la comparaison et le PC est mis à jour selon le résultat.

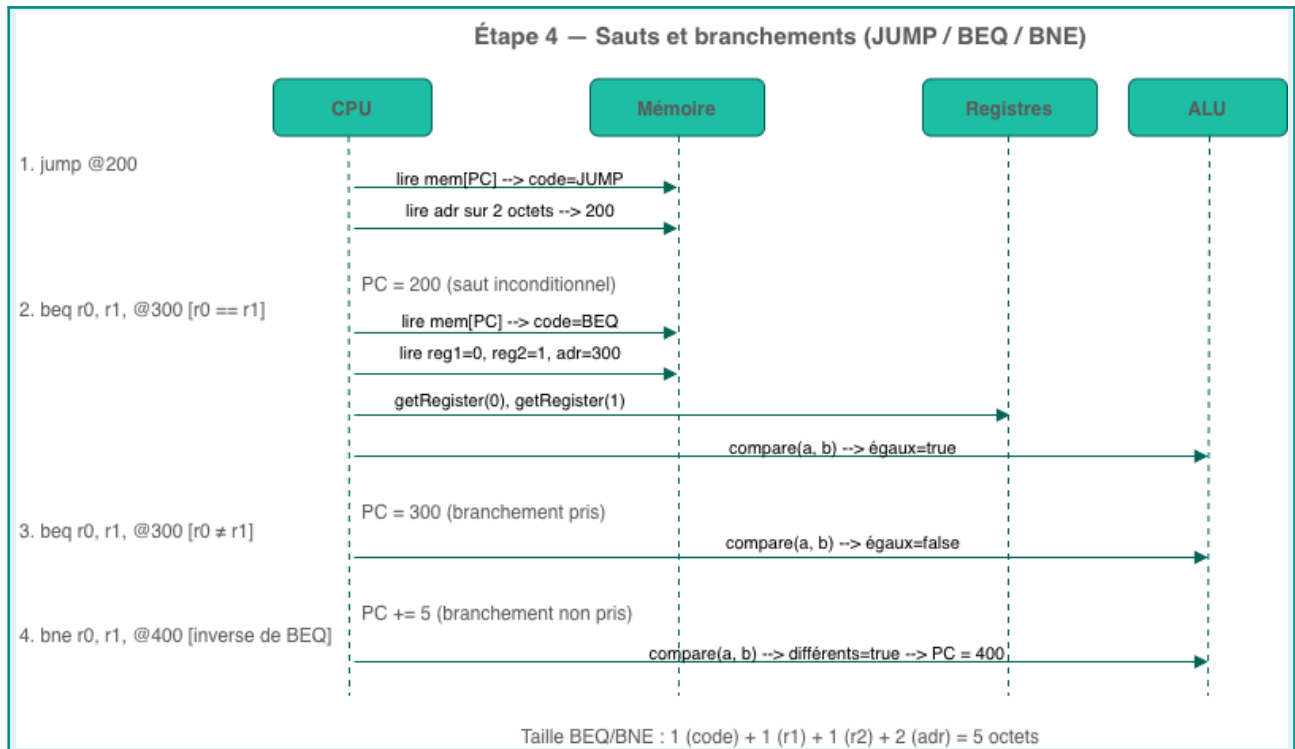


Figure 6 — Séquence Étape 4 : Sauts et branchements

3.5 Étape 5 — Load/Store indexé, DATA et STRING

Ce dernier diagramme couvre les instructions avancées : `LOAD_INDEXE` et `STORE_INDEXE` (accès mémoire avec adresse de base + offset depuis un registre), ainsi que les directives assembleur `DATA` (initialisation de données numériques) et `STRING` (écriture de chaînes UTF-8 en mémoire). Ces directives sont traitées par l'Assembleur et non exécutées par le CPU.

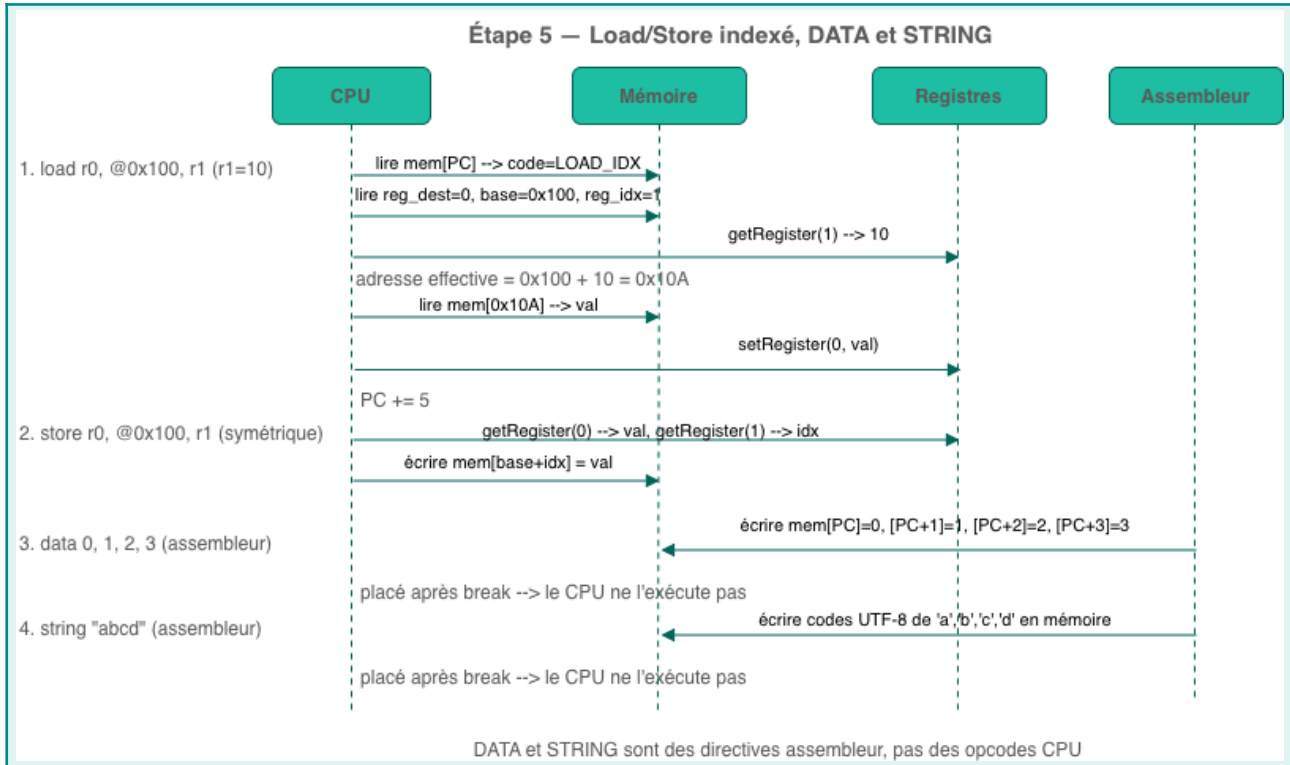


Figure 7 — Séquence Étape 5 : Load/Store indexé, DATA et STRING